

INTELLIGENT PATIENT FLOW AND APPOINTMENT SCHEDULING SYSTEM

Database Design and Technical Documentation

PostgreSQL 15+ | Django Modular Monolith Baseline

Version 1.0

July 2026

CONFIDENTIAL COMMERCIAL DESIGN DOCUMENT

Document Control

Document title	Intelligent Patient Flow and Appointment Scheduling System - Database Design and Technical Documentation
Database platform	PostgreSQL 15 or later
Application architecture	Django modular monolith with separate web and mobile clients
Schema version	1.0
Status	Commercial implementation baseline

Table of Contents

Table of Contents

Document Control.....	1
Table of Contents.....	2
1. Executive Summary.....	6
1.1 Business Problems Solved.....	6
1.2 Commercial Capabilities Enabled.....	6
2. Design Principles and Corrected Decisions.....	7
3. Technical Architecture.....	8
3.1 Naming and Data-Type Conventions.....	8
3.2 Module Inventory.....	8
3.3 Complete Table Inventory.....	10
4. Detailed Data Dictionary.....	13
4.1 Organization and Facility Structure.....	13
organizations.....	13
facility_types.....	14
facilities.....	15
departments.....	18
specialties.....	19
facility_specialties.....	20
service_point_types.....	22
service_points.....	23
consultation_rooms.....	25
facility_operating_hours.....	26
facility_schedule_exceptions.....	28

facility_flow_settings.....	29
4.2 Identity, Membership, Roles and Permissions.....	31
users.....	31
roles.....	33
permissions.....	35
role_permissions.....	36
user_memberships.....	37
user_role_assignments.....	39
4.3 Patient Registry and Delegated Access.....	40
patients.....	40
patient_identifier_types.....	42
patient_identifiers.....	44
patient_addresses.....	46
relationship_types.....	48
patient_related_persons.....	49
related_person_contacts.....	51
patient_access_grants.....	52
4.4 Practitioner Registry and Professional Assignments.....	54
practitioner_types.....	54
practitioners.....	56
practitioner_facility_assignments.....	57
practitioner_department_assignments.....	59
practitioner_specialty_assignments.....	60
practitioner_credential_types.....	62
practitioner_credentials.....	63
4.5 Scheduling and Appointment Management.....	65

practitioner_availability_periods.....	65
practitioner_leave_requests.....	67
practitioner_shifts.....	69
appointments.....	71
appointment_status_history.....	74
appointment_slots.....	75
4.6 Patient Arrival and Secure Check-in.....	76
patient_checkins.....	77
checkin_tokens.....	78
4.7 Queue Operations.....	80
queues.....	80
queue_entries.....	82
queue_transfers.....	84
queue_entry_events.....	85
4.8 Intelligence and Prediction.....	86
queue_wait_time_predictions.....	86
4.9 Patient Communication.....	88
patient_notifications.....	88
user_push_devices.....	90
4.10 Audit and Reporting.....	92
audit_logs.....	92
report_exports.....	93
5. Critical Transaction Workflows.....	96
Appointment booking.....	96
Appointment rescheduling.....	96
QR check-in.....	96

Queue entry creation.....	96
Queue service progression.....	96
Queue transfer.....	97
Notification delivery.....	97
6. Security, Privacy, and Audit.....	97
6.1 Sensitive Data Protection.....	97
6.2 Authorization Boundaries.....	97
6.3 Audit and Immutability.....	98
7. Performance and Reporting Strategy.....	98
8. Django and Deployment Guidance.....	98
8.1 Recommended Application Modules.....	99
9. Deliberately Excluded Tables.....	99
10. Conclusion.....	100
Appendix A. Source SQL Script.....	100

1. Executive Summary

This database is the operational foundation of a patient-flow and appointment-scheduling platform. It is designed for healthcare organizations that operate one or more facilities and need a single controlled source of truth for patient registration, practitioner schedules, appointment booking, arrival, queue management, communication, prediction, audit, and reporting.

The design uses 50 business tables. Each table exists because it represents an entity, relationship, workflow, security boundary, or historical event with an independent lifecycle. The schema deliberately avoids both extremes: it does not place all data in a few oversized tables, and it does not create separate tables for values that can be stored cleanly on their owner record.

1.1 Business Problems Solved

- Fragmented facility, department, specialty, room, and service-point information.
- Manual or conflicting practitioner availability, leave, shift, and room allocation.
- Appointment double booking, unclear rescheduling history, and inconsistent status transitions.
- Slow or insecure patient arrival procedures, including QR-code check-in.
- Uncontrolled daily queues, duplicate queue numbers, unclear priority handling, and lost transfer history.
- Inability to estimate and evaluate patient waiting time.
- Hardcoded staff roles and permissions that cannot adapt to different healthcare customers.
- Weak traceability for sensitive operations, report generation, and patient-specific delegated access.
- Duplicate notifications, exposed contact details, and poor delivery tracking.

1.2 Commercial Capabilities Enabled

- Multi-organization and multi-facility deployment with strict tenant scope.
- Dynamic role-based access control at platform, organization, facility, and individual-patient scope.
- Appointment booking through web, mobile, reception, or external API.
- Walk-in and appointment patient check-in using reception, mobile, kiosk, or secure QR code.
- Priority-aware queue processing, repeated calls, service timing, and inter-queue transfer.
- Rule-based and machine-learning waiting-time prediction with model-version history.
- Encrypted patient identifiers, practitioner credentials, addresses, notifications, and device tokens.
- Operational dashboards and exportable reports derived from trustworthy transactional data.

2. Design Principles

The final design incorporates several corrections made during the design review. These decisions reduce complexity while strengthening data integrity.

Reviewed area	Final decision
Patient and practitioner contacts	Separate <code>patient_contacts</code> and <code>practitioner_contacts</code> tables were removed. One operational phone and email are stored directly on patients and practitioners. A separate contact table remains only for related persons, where multiple contacts are a real requirement.
Queue-number prefix	The queue prefix is derived from <code>service_points.code</code> . It is not duplicated in queues.
Room allocation	<code>consultation_room_id</code> is stored directly on <code>practitioner_shifts</code> because the current product uses one room per shift segment. A separate room-allocation table was intentionally avoided.
Django password field	The <code>users</code> table uses <code>password</code> rather than <code>password_hash</code> so it remains compatible with Django <code>AbstractBaseUser</code> while still storing only encoded hashes.
Membership scope	<code>user_memberships</code> always requires <code>organization_id</code> . <code>facility_id</code> is optional and must belong to that organization.
Practitioner booking validation	An appointment with a practitioner must use a valid <code>practitioner_specialty_assignment</code> , preventing a practitioner from being booked for an unauthorized specialty.
Cross-facility conflict prevention	The database prevents overlapping practitioner shifts and appointments even when different facility assignments are involved.
Derived values	Queue position, displayed queue number, actual waiting time, dashboard totals, and practitioner workload summaries are derived rather than stored redundantly.
History protection	Appointment history, queue events, predictions,

Reviewed area	Final decision
	and audit logs are append-only.

3. Technical Architecture

The schema is designed for PostgreSQL 15 or later and a Django modular monolith. PostgreSQL extensions pgcrypto, citext, and btree_gist support UUID generation, case-insensitive values, and exclusion constraints. Application services perform multi-table workflows inside transactions, while database constraints and triggers provide a final integrity boundary.

3.1 Naming and Data-Type Conventions

- Table names use plural snake_case; Django models use singular PascalCase.
- Primary keys use UUID to avoid predictable public identifiers and to support distributed creation.
- Foreign keys use the <entity>_id pattern.
- TIMESTAMPTZ is used for real instants; DATE and TIME are used only for local recurring schedules.
- CITEXT is used where case-insensitive uniqueness is required, especially emails and controlled names.
- Sensitive values are encrypted by the application. Exact-match lookup uses keyed HMAC-SHA-256, never ordinary unsalted hashing.
- ON DELETE RESTRICT/PROTECT preserves operational history; CASCADE is used only for true dependent configuration such as facility settings.

3.2 Module Inventory

Module	Tables	Responsibility
Organization and Facility Structure	12	Defines tenancy, physical locations, facility configuration, clinical offerings, rooms, service points, opening hours, and local workflow policies.
Identity, Membership, Roles and Permissions	6	Separates authentication from organizational membership and dynamic authorization so each

Module	Tables	Responsibility
		customer can configure its own roles without code changes.
Patient Registry and Delegated Access	8	Maintains patient identities, sensitive identifiers, addresses, related persons, and patient-specific delegated access.
Practitioner Registry and Professional Assignments	7	Maintains practitioner profiles, work locations, departments, specialties, professional credentials, and verification status.
Scheduling and Appointment Management	6	Controls recurring availability, leave, shifts, rooms, bookable slots, appointment conflicts, rescheduling, and lifecycle history.
Patient Arrival and Secure Check-in	2	Records physical or digital arrival and secures QR check-in through expiring single-use tokens.
Queue Operations	4	Manages daily queues, priority-aware patient entries, calls, service timestamps, transfer chains, and immutable event history.
Intelligence and Prediction	1	Stores repeated waiting-time estimates and model versions so prediction quality can be evaluated against actual waits.
Patient Communication	2	Schedules and tracks encrypted multi-channel notifications and manages push-notification devices.
Audit and Reporting	2	Provides immutable platform audit evidence and tracks secure asynchronous report generation.

3.3 Complete Table Inventory

No.	Table	Module
1	organizations	Organization and Facility Structure
2	facility_types	Organization and Facility Structure
3	facilities	Organization and Facility Structure
4	departments	Organization and Facility Structure
5	specialties	Organization and Facility Structure
6	facility_specialties	Organization and Facility Structure
7	service_point_types	Organization and Facility Structure
8	service_points	Organization and Facility Structure
9	consultation_rooms	Organization and Facility Structure
10	facility_operating_hours	Organization and Facility Structure
11	facility_schedule_exceptions	Organization and Facility Structure
12	facility_flow_settings	Organization and Facility Structure
13	users	Identity, Membership, Roles and Permissions
14	roles	Identity, Membership, Roles and Permissions
15	permissions	Identity, Membership, Roles and Permissions

No.	Table	Module
16	role_permissions	Identity, Membership, Roles and Permissions
17	user_memberships	Identity, Membership, Roles and Permissions
18	user_role_assignments	Identity, Membership, Roles and Permissions
19	patients	Patient Registry and Delegated Access
20	patient_identifier_types	Patient Registry and Delegated Access
21	patient_identifiers	Patient Registry and Delegated Access
22	patient_addresses	Patient Registry and Delegated Access
23	relationship_types	Patient Registry and Delegated Access
24	patient_related_persons	Patient Registry and Delegated Access
25	related_person_contacts	Patient Registry and Delegated Access
26	patient_access_grants	Patient Registry and Delegated Access
27	practitioner_types	Practitioner Registry and Professional Assignments
28	practitioners	Practitioner Registry and Professional Assignments
29	practitioner_facility_assignments	Practitioner Registry and Professional Assignments
30	practitioner_department_assignments	Practitioner Registry and Professional Assignments
31	practitioner_specialty_assignments	Practitioner Registry and Professional Assignments

No.	Table	Module
32	practitioner_credential_types	Practitioner Registry and Professional Assignments
33	practitioner_credentials	Practitioner Registry and Professional Assignments
34	practitioner_availability_periods	Scheduling and Appointment Management
35	practitioner_leave_requests	Scheduling and Appointment Management
36	practitioner_shifts	Scheduling and Appointment Management
37	appointments	Scheduling and Appointment Management
38	appointment_status_history	Scheduling and Appointment Management
39	appointment_slots	Scheduling and Appointment Management
40	patient_checkins	Patient Arrival and Secure Check-in
41	checkin_tokens	Patient Arrival and Secure Check-in
42	queues	Queue Operations
43	queue_entries	Queue Operations
44	queue_transfers	Queue Operations
45	queue_entry_events	Queue Operations
46	queue_wait_time_predictions	Intelligence and Prediction
47	patient_notifications	Patient Communication
48	user_push_devices	Patient Communication
49	audit_logs	Audit and Reporting
50	report_exports	Audit and Reporting

4. Detailed Data Dictionary

This section documents every business table. The field definitions are taken from the finalized PostgreSQL script. “Why standalone” explains the normalization or lifecycle reason for keeping the table independent.

4.1 Organization and Facility Structure

Defines tenancy, physical locations, facility configuration, clinical offerings, rooms, service points, opening hours, and local workflow policies.

organizations

Purpose. Represents the commercial customer or healthcare organization that owns one or more facilities and provides the top-level tenant boundary.

Why this table is standalone. It has its own lifecycle, legal identity, code, and multiple dependent facilities. Embedding it in facilities would duplicate tenant information and weaken isolation.

Key relationships. Referenced by audit_logs, facilities, patient_identifier_types, patients, practitioner_credential_types, practitioners, report_exports, roles, user_memberships.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
name	VARCHAR(150)	Required	Human-readable name.
legal_name	VARCHAR(200)	Optional	Registered legal name of the organization.
code	VARCHAR(30)	Required	Stable business code used for identification and integration.
email	CITEXT	Optional	Normalized email address.
phone_number	VARCHAR(30)	Optional	Normalized E.164 phone number.

Column	Type	Required / Default / Key	Business meaning
registration_number	VARCHAR(100)	Optional	Business or sequence number for registration.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table organizations: 10 columns in the finalized SQL schema.

Key integrity and operational rules:

- Organization codes are unique and uppercase.
- Phone numbers use E.164 format when provided.
- Inactive organizations remain in history instead of being physically deleted.

facility_types

Purpose. Defines reusable classifications for facilities, such as hospital, clinic, diagnostic centre, or dispensary.

Why this table is standalone. Many facilities reuse the same classification. A reference table avoids inconsistent free-text values and allows administrators to manage types centrally.

Key relationships. Referenced by facilities.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.

Column	Type	Required / Default / Key	Business meaning
name	CITEXT	Required	Human-readable name.
code	VARCHAR(30)	Required	Stable business code used for identification and integration.
description	TEXT	Optional	Optional explanatory text for administrators and operators.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table facility_types: 7 columns in the finalized SQL schema.

Key integrity and operational rules:

- Names and codes are globally unique.
- Codes are uppercase stable identifiers.
- Types are dynamic; no facility category is hardcoded in application logic.

facilities

Purpose. Stores each physical healthcare location operated by an organization, including contact, address, timezone, licensing, and location data.

Why this table is standalone. A facility has independent licensing, location, timezone, operating rules, departments, queues, and reports, so it must be a first-class entity.

Key relationships. References facility_types, organizations. Referenced by appointments, audit_logs, consultation_rooms, departments, facility_flow_settings, facility_operating_hours,

facility_schedule_exceptions, facility_specialties, patient_checkins, patients,
 practitioner_facility_assignments, report_exports, roles, service_points, user_memberships.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
organization_id	UUID	Required; FK to organizations; ON DELETE RESTRICT	References the related organizations record.
facility_type_id	UUID	Required; FK to facility_types; ON DELETE RESTRICT	References the related facility types record.
name	VARCHAR(150)	Required	Human-readable name.
code	VARCHAR(30)	Required	Stable business code used for identification and integration.
license_number	VARCHAR(100)	Optional	Business or sequence number for license.
email	CITEXT	Optional	Normalized email address.
phone_number	VARCHAR(30)	Optional	Normalized E.164 phone number.
address_line1	VARCHAR(200)	Optional	Address line1.
address_line2	VARCHAR(200)	Optional	Address line2.
country_code	CHAR(2)	Optional	Country code.
region	VARCHAR(100)	Optional	Region.
district	VARCHAR(100)	Optional	District.
ward	VARCHAR(100)	Optional	Ward.
postal_code	VARCHAR(20)	Optional	Postal code.
latitude	NUMERIC(9,6)	Optional	Latitude.

Column	Type	Required / Default / Key	Business meaning
longitude	NUMERIC(10,7)	Optional	Longitude.
timezone	VARCHAR(64)	Required; Default: 'Africa/Dar_es_Salaam'	IANA timezone used to interpret facility-local operating dates and times.
is_primary	BOOLEAN	Required; Default: FALSE	Boolean flag indicating whether primary applies.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table facilities: 22 columns in the finalized SQL schema.

Key integrity and operational rules:

- Facility code is unique within its organization.
- Only one active primary facility is allowed per organization.
- Coordinates must be supplied as a valid latitude/longitude pair.
- Facility timezone controls local dates for schedules and queues.

Indexing approach. The SQL script defines 4 explicit index(es) for this table, including 1 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

departments

Purpose. Represents the organizational units within a facility and supports hierarchical structures such as Outpatient > Pediatrics.

Why this table is standalone. Departments form a reusable hierarchy and are referenced by service points, rooms, specialties, and practitioner assignments.

Key relationships. References departments, facilities. Referenced by consultation_rooms, departments, facility_specialties, practitioner_department_assignments, service_points.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
facility_id	UUID	Required; FK to facilities; ON DELETE RESTRICT	References the related facilities record.
parent_department_id	UUID	Optional; FK to departments; ON DELETE RESTRICT	References the related departments record.
name	VARCHAR(150)	Required	Human-readable name.
code	VARCHAR(30)	Required	Stable business code used for identification and integration.
description	TEXT	Optional	Optional explanatory text for administrators and operators.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.

Column	Type	Required / Default / Key	Business meaning
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table departments: 9 columns in the finalized SQL schema.

Key integrity and operational rules:

- Department codes are unique within a facility.
- A parent department must belong to the same facility.
- Self-parenting and hierarchy cycles are rejected.

Indexing approach. The SQL script defines 2 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

specialties

Purpose. Defines the global clinical specialty catalogue used to classify services and practitioner expertise.

Why this table is standalone. Clinical specialties are shared across organizations and must not be duplicated separately in every facility.

Key relationships. References specialties. Referenced by facility_specialties, specialties.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
parent_specialty_id	UUID	Optional; FK to specialties; ON DELETE RESTRICT	References the related specialties record.
name	CITEXT	Required	Human-readable name.
code	VARCHAR(30)	Required	Stable business code used for identification

Column	Type	Required / Default / Key	Business meaning
			and integration.
description	TEXT	Optional	Optional explanatory text for administrators and operators.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table specialties: 8 columns in the finalized SQL schema.

Key integrity and operational rules:

- Names and codes are globally unique.
- Specialty hierarchies cannot contain cycles.
- Global specialty records contain no facility-specific booking settings.

Indexing approach. The SQL script defines 1 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

facility_specialties

Purpose. Links a specialty to a facility or department and stores local booking and walk-in rules for that offering.

Why this table is standalone. The global specialty definition does not contain local duration, walk-in, referral, or department settings. This junction carries those facility-specific rules.

Key relationships. References departments, facilities, specialties. Referenced by appointment_slots, appointments, patient_checkins, practitioner_specialty_assignments, queues.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
facility_id	UUID	Required; FK to facilities; ON DELETE RESTRICT	References the related facilities record.
specialty_id	UUID	Required; FK to specialties; ON DELETE RESTRICT	References the related specialties record.
department_id	UUID	Optional; FK to departments; ON DELETE RESTRICT	References the related departments record.
appointment_duration_minutes	SMALLINT	Required	Appointment duration minutes.
accepts_appointments	BOOLEAN	Required; Default: TRUE	Boolean flag controlling whether the record accepts appointments.
accepts_walk_ins	BOOLEAN	Required; Default: FALSE	Boolean flag controlling whether the record accepts walk ins.
requires_referral	BOOLEAN	Required; Default: FALSE	Boolean flag indicating whether referral is required.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.

Column	Type	Required / Default / Key	Business meaning
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table *facility_specialties*: 11 columns in the finalized SQL schema.

Key integrity and operational rules:

- Department, when present, must belong to the selected facility.
- Duplicate facility/specialty offerings are prevented for both department and non-department cases.
- Appointment duration must be positive.

Indexing approach. The SQL script defines 5 explicit index(es) for this table, including 2 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

service_point_types

Purpose. Defines reusable operational categories for service points such as reception, laboratory desk, pharmacy counter, or consultation point.

Why this table is standalone. Operational categories are reused by many service points and should not be stored as unrestricted text.

Key relationships. Referenced by service_points.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
name	CITEXT	Required	Human-readable name.
code	VARCHAR(30)	Required	Stable business code used for identification

Column	Type	Required / Default / Key	Business meaning
			and integration.
description	TEXT	Optional	Optional explanatory text for administrators and operators.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table service_point_types: 7 columns in the finalized SQL schema.

Key integrity and operational rules:

- Names and codes are unique.
- Codes are uppercase.
- Operational behaviour is not inferred from the code alone.

service_points

Purpose. Represents the operational locations where patients join queues or receive non-room services inside a facility.

Why this table is standalone. A service point has its own queue history, code, location, department, and activation lifecycle.

Key relationships. References departments, facilities, service_point_types. Referenced by practitioner_shifts, queues.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
facility_id	UUID	Required; FK to facilities; ON DELETE RESTRICT	References the related facilities record.
department_id	UUID	Optional; FK to departments; ON DELETE RESTRICT	References the related departments record.
service_point_type_id	UUID	Required; FK to service_point_types; ON DELETE RESTRICT	References the related service point types record.
name	VARCHAR(150)	Required	Human-readable name.
code	VARCHAR(30)	Required	Stable business code used for identification and integration.
location_description	VARCHAR(250)	Optional	Location description.
floor	VARCHAR(30)	Optional	Floor.
display_order	INTEGER	Required; Default: 0	Display order.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table service_points: 12 columns in the finalized SQL schema.

Key integrity and operational rules:

- Codes are unique within a facility.
- The optional department must belong to the same facility.
- Display order cannot be negative.

Indexing approach. The SQL script defines 3 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

consultation_rooms

Purpose. Represents physical rooms that can be allocated to practitioner shifts and protected against conflicting use.

Why this table is standalone. Rooms are physical resources with independent capacity and conflict rules; they are not equivalent to service points.

Key relationships. References departments, facilities. Referenced by practitioner_shifts.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
facility_id	UUID	Required; FK to facilities; ON DELETE RESTRICT	References the related facilities record.
department_id	UUID	Optional; FK to departments; ON DELETE RESTRICT	References the related departments record.
name	VARCHAR(150)	Required	Human-readable name.
code	VARCHAR(30)	Required	Stable business code used for identification and integration.
location_description	VARCHAR(250)	Optional	Location description.
floor	VARCHAR(30)	Optional	Floor.
capacity	SMALLINT	Required; Default: 1	Capacity.

Column	Type	Required / Default / Key	Business meaning
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table consultation_rooms: 11 columns in the finalized SQL schema.

Key integrity and operational rules:

- Room codes are unique within a facility.
- Room capacity must be positive.
- Room department must match the facility and room allocations cannot overlap.

Indexing approach. The SQL script defines 2 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

facility_operating_hours

Purpose. Stores the recurring weekly opening periods of a facility, including split shifts and overnight operation.

Why this table is standalone. A facility can have several periods per day. This one-to-many schedule cannot be represented safely by columns on facilities.

Key relationships. References facilities.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid();	System-generated UUID primary key.

Column	Type	Required / Default / Key	Business meaning
		Primary key	
facility_id	UUID	Required; FK to facilities; ON DELETE CASCADE	References the related facilities record.
day_of_week	SMALLINT	Required	Facility-local weekday number from 1 (Monday) to 7 (Sunday).
period_order	SMALLINT	Required; Default: 1	Ordering number for multiple opening periods on the same day.
opens_at	TIME	Optional	Timestamp for opens.
closes_at	TIME	Optional	Timestamp for closes.
closes_next_day	BOOLEAN	Required; Default: FALSE	Closes next day.
is_24_hours	BOOLEAN	Required; Default: FALSE	Boolean flag indicating whether 24 hours applies.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table facility_operating_hours: 11 columns in the finalized SQL schema.

Key integrity and operational rules:

- Day is limited to Monday through Sunday represented by 1-7.
- 24-hour, normal, and overnight periods use mutually consistent time fields.
- Active periods cannot overlap.

Indexing approach. The SQL script defines 1 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

facility_schedule_exceptions

Purpose. Overrides normal operating hours for a specific date, including holidays, closures, extended hours, and emergency schedules.

Why this table is standalone. Date-specific overrides have a different lifecycle from recurring weekly hours and must preserve holiday or emergency changes independently.

Key relationships. References facilities.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
facility_id	UUID	Required; FK to facilities; ON DELETE CASCADE	References the related facilities record.
exception_date	DATE	Required	Facility-local calendar date whose normal schedule is overridden.
period_order	SMALLINT	Required; Default: 1	Period order.
is_closed	BOOLEAN	Required; Default: FALSE	Boolean flag indicating whether closed applies.
opens_at	TIME	Optional	Timestamp for opens.
closes_at	TIME	Optional	Timestamp for closes.
closes_next_day	BOOLEAN	Required; Default: FALSE	Closes next day.
is_24_hours	BOOLEAN	Required; Default:	Boolean flag indicating

Column	Type	Required / Default / Key	Business meaning
		FALSE	whether 24 hours applies.
reason	VARCHAR(250)	Optional	Reason or explanation for the record or transition.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table *facility_schedule_exceptions*: 13 columns in the finalized SQL schema.

Key integrity and operational rules:

- One date may be closed, 24-hour, or contain non-overlapping custom periods.
- Date exceptions override normal weekly hours.
- Closed or 24-hour records cannot coexist with other active periods for that date.

Indexing approach. The SQL script defines 1 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

facility_flow_settings

Purpose. Stores facility-specific booking, check-in, reminder, no-show, and queue-number behaviour so those rules are configurable rather than hardcoded.

Why this table is standalone. These are one-per-facility behavioural settings. Separating them keeps the facility master record stable and prevents unrelated configuration columns from growing indefinitely.

Key relationships. References facilities, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
facility_id	UUID	Required; Unique; FK to facilities; ON DELETE CASCADE	References the related facilities record.
max_advance_booking_days	SMALLINT	Required; Default: 30	Max advance booking days.
minimum_booking_notice_minutes	INTEGER	Required; Default: 0	Minimum booking notice minutes.
cancellation_cutoff_minutes	INTEGER	Required; Default: 60	Cancellation cutoff minutes.
reschedule_cutoff_minutes	INTEGER	Required; Default: 60	Reschedule cutoff minutes.
early_checkin_minutes	INTEGER	Required; Default: 30	Early checkin minutes.
late_checkin_grace_minutes	INTEGER	Required; Default: 15	Late checkin grace minutes.
no_show_after_minutes	INTEGER	Required; Default: 15	No show after minutes.
default_reminder_minutes_before	INTEGER	Optional; Default: 1440	Default reminder minutes before.
queue_number_padding	SMALLINT	Required; Default: 3	Number of digits used when formatting queue sequence numbers.
auto_create_daily_queues	BOOLEAN	Required; Default: FALSE	Auto create daily queues.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the

Column	Type	Required / Default / Key	Business meaning
		MP	record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP MP	Timestamp of the most recent update; maintained automatically where applicable.

Table *facility_flow_settings*: 15 columns in the finalized SQL schema.

Key integrity and operational rules:

- One settings row exists per facility.
- All duration and notice values are non-negative.
- Queue number padding is limited to 1-6 digits.

4.2 Identity, Membership, Roles and Permissions

Separates authentication from organizational membership and dynamic authorization so each customer can configure its own roles without code changes.

users

Purpose. Stores authenticated system identities used by patients, staff, administrators, and practitioners.

Why this table is standalone. Authentication identity is distinct from patient and practitioner profiles because one account may act in several business capacities.

Key relationships. Referenced by appointment_status_history, appointments, audit_logs, checkin_tokens, facility_flow_settings, patient_access_grants, patient_addresses, patient_checkins, patient_identifier_types, patient_identifiers, patient_notifications, patient_related_persons, patients, permissions, practitioner_availability_periods, practitioner_credential_types, practitioner_credentials, practitioner_department_assignments, practitioner_facility_assignments, practitioner_leave_requests, practitioner_shifts, practitioner_specialty_assignments, practitioner_types, practitioners, queue_entries, queue_entry_events, queue_transfers, queues, related_person_contacts, relationship_types, report_exports, role_permissions, roles, user_memberships, user_push_devices, user_role_assignments.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
email	CITEXT	Optional	Normalized email address.
phone_number	VARCHAR(30)	Optional	Normalized E.164 phone number.
password	VARCHAR(128)	Required	Django-compatible encoded password value; never stores plaintext.
first_name	VARCHAR(100)	Required	First name.
middle_name	VARCHAR(100)	Optional	Middle name.
last_name	VARCHAR(100)	Required	Last name.
email_verified_at	TIMESTAMPTZ	Optional	Timestamp for email verified.
phone_verified_at	TIMESTAMPTZ	Optional	Timestamp for phone verified.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
is_staff	BOOLEAN	Required; Default: FALSE	Boolean flag indicating whether staff applies.
is_superuser	BOOLEAN	Required; Default: FALSE	Boolean flag indicating whether superuser applies.
last_login	TIMESTAMPTZ	Optional	Last login.
date_joined	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Date joined.

Column	Type	Required / Default / Key	Business meaning
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table users: 16 columns in the finalized SQL schema.

Key integrity and operational rules:

- At least one of email or phone number is required.
- Email and phone are independently unique when present.
- Verification is represented by timestamps.
- is_staff is only for Django admin access; business access comes from dynamic roles.

Indexing approach. The SQL script defines 2 explicit index(es) for this table, including 2 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

roles

Purpose. Defines dynamic platform-, organization-, or facility-scoped business roles without hardcoding role names in application logic.

Why this table is standalone. Roles are configurable business objects with scope and dates; hardcoding them would prevent commercial customization.

Key relationships. References facilities, organizations, users. Referenced by patient_access_grants, role_permissions, user_role_assignments.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
organization_id	UUID	Optional; FK to	Organization scope of

Column	Type	Required / Default / Key	Business meaning
		organizations; ON DELETE RESTRICT	the role; null for a platform-level role.
facility_id	UUID	Optional; FK to facilities; ON DELETE RESTRICT	Facility scope of the role; null for platform or organization scope.
name	VARCHAR(100)	Required	Human-readable name.
code	VARCHAR(80)	Required	Stable business code used for identification and integration.
description	TEXT	Optional	Optional explanatory text for administrators and operators.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table roles: 10 columns in the finalized SQL schema.

Key integrity and operational rules:

- A role is platform-, organization-, or facility-scoped.
- Facility-scoped roles must reference a facility inside the same organization.
- Role names and codes are unique within their scope.

Indexing approach. The SQL script defines 8 explicit index(es) for this table, including 6 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

permissions

Purpose. Defines the reusable atomic actions that may be granted to roles, such as appointment.create or queue.call.

Why this table is standalone. Permissions are reusable atomic actions and must exist independently so different roles can share them.

Key relationships. References users. Referenced by role_permissions.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
name	CITEXT	Required	Human-readable name.
code	VARCHAR(120)	Required	Stable business code used for identification and integration.
module	VARCHAR(60)	Required	Functional module containing the permitted action.
action	VARCHAR(60)	Required	Atomic action name within the module.
description	TEXT	Optional	Optional explanatory text for administrators and operators.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_by_id	UUID	Optional; FK to users; ON DELETE SET	User who created the record, retained as

Column	Type	Required / Default / Key	Business meaning
		NULL	nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table permissions: 10 columns in the finalized SQL schema.

Key integrity and operational rules:

- Permission code is globally unique and uses module.action form.
- Module/action combinations are unique.
- Permissions remain independent of any role or facility.

role_permissions

Purpose. Implements the many-to-many relationship between roles and permissions.

Why this table is standalone. The relationship is many-to-many and has its own active state and grant metadata.

Key relationships. References permissions, roles, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
role_id	UUID	Required; FK to roles; ON DELETE RESTRICT	References the related roles record.
permission_id	UUID	Required; FK to permissions; ON DELETE RESTRICT	References the related permissions record.

Column	Type	Required / Default / Key	Business meaning
granted_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table *role_permissions*: 7 columns in the finalized SQL schema.

Key integrity and operational rules:

- One active relationship exists per role/permission pair.
- Inactive links may be reactivated instead of duplicated.
- Effective authorization also requires active role and permission records.

Indexing approach. The SQL script defines 2 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

user_memberships

Purpose. Records where a user belongs organizationally, either at organization level or at a specific facility.

Why this table is standalone. Membership answers where a user belongs; role assignment answers what the user may do. Combining them would mix organizational scope with authorization.

Key relationships. References facilities, organizations, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
user_id	UUID	Required; FK to users; ON DELETE RESTRICT	References the related users record.
organization_id	UUID	Required; FK to organizations; ON DELETE RESTRICT	References the related organizations record.
facility_id	UUID	Optional; FK to facilities; ON DELETE RESTRICT	References the related facilities record.
starts_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Start date and time of the period.
ends_at	TIMESTAMPTZ	Optional	End date and time of the period.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table user_memberships: 10 columns in the finalized SQL schema.

Key integrity and operational rules:

- Organization is always required; facility is optional.
- A facility membership must match its organization.
- Date ranges must be valid and duplicate active memberships are prevented.

Indexing approach. The SQL script defines 5 explicit index(es) for this table, including 2 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

user_role_assignments

Purpose. Assigns roles to users for a controlled time period after validating that the user has a matching membership.

Why this table is standalone. Assignments have independent effective dates, creator, and lifecycle, and one user may hold several roles.

Key relationships. References roles, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
user_id	UUID	Required; FK to users; ON DELETE RESTRICT	References the related users record.
role_id	UUID	Required; FK to roles; ON DELETE RESTRICT	References the related roles record.
assigned_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
starts_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Start date and time of the period.
ends_at	TIMESTAMPTZ	Optional	End date and time of the period.

Column	Type	Required / Default / Key	Business meaning
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table user_role_assignments: 9 columns in the finalized SQL schema.

Key integrity and operational rules:

- The user must have an active membership matching the role scope.
- Assignment dates must be valid.
- Duplicate user/role assignments are prevented.

Indexing approach. The SQL script defines 2 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

4.3 Patient Registry and Delegated Access

Maintains patient identities, sensitive identifiers, addresses, related persons, and patient-specific delegated access.

patients

Purpose. Stores the organization-specific patient master record used across booking, check-in, queueing, notifications, and reporting.

Why this table is standalone. The patient record is the central clinical-flow identity and must exist even when the patient has no login account.

Key relationships. References facilities, organizations, users. Referenced by appointments, patient_access_grants, patient_addresses, patient_checkins, patient_identifiers, patient_notifications, patient_related_persons.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
organization_id	UUID	Required; FK to organizations; ON DELETE RESTRICT	References the related organizations record.
user_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
registered_facility_id	UUID	Optional; FK to facilities; ON DELETE RESTRICT	References the related facilities record.
patient_number	VARCHAR(50)	Required	Organization-specific human-readable patient identifier.
first_name	VARCHAR(100)	Required	First name.
middle_name	VARCHAR(100)	Optional	Middle name.
last_name	VARCHAR(100)	Required	Last name.
date_of_birth	DATE	Optional	Date of birth.
date_of_birth_is_estimated	BOOLEAN	Required; Default: FALSE	Indicates that the date of birth is approximate rather than confirmed.
sex_code	VARCHAR(20)	Optional	Sex code.
email	CITEXT	Optional	Normalized email address.
phone_number	VARCHAR(30)	Optional	Normalized E.164 phone number.
is_active	BOOLEAN	Required; Default:	Controls whether the

Column	Type	Required / Default / Key	Business meaning
		TRUE	record is currently available for operational use without deleting history.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table patients: 17 columns in the finalized SQL schema.

Key integrity and operational rules:

- Patient number is unique within an organization.
- A linked user may have only one patient profile per organization.
- Registered facility must belong to the patient organization.
- Phone and email are normalized but not globally unique.

Indexing approach. The SQL script defines 4 explicit index(es) for this table, including 1 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

patient_identifier_types

Purpose. Defines configurable identifier categories such as national ID, insurance number, passport, or hospital card number.

Why this table is standalone. Identifier categories are configurable by organization and carry sensitivity and expiry expectations.

Key relationships. References organizations, users. Referenced by patient_identifiers.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
organization_id	UUID	Optional; FK to organizations; ON DELETE RESTRICT	References the related organizations record.
name	VARCHAR(100)	Required	Human-readable name.
code	VARCHAR(40)	Required	Stable business code used for identification and integration.
description	TEXT	Optional	Optional explanatory text for administrators and operators.
is_sensitive	BOOLEAN	Required; Default: TRUE	Boolean flag indicating whether sensitive applies.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table patient_identifier_types: 10 columns in the finalized SQL schema.

Key integrity and operational rules:

- Identifier types may be global or organization-specific.
- Codes and names are unique within the applicable scope.
- Sensitivity and expiry requirements are metadata, not hardcoded behaviour.

Indexing approach. The SQL script defines 5 explicit index(es) for this table, including 4 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

patient_identifiers

Purpose. Stores encrypted patient identifier values and verification information separately from the basic patient profile.

Why this table is standalone. Identifiers are sensitive, repeatable, verifiable, and independently expirable, so they should not be mixed into the basic patient row.

Key relationships. References patient_identifier_types, patients, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
patient_id	UUID	Required; FK to patients; ON DELETE RESTRICT	References the related patients record.
identifier_type_id	UUID	Required; FK to patient_identifier_types; ON DELETE RESTRICT	References the related patient identifier types record.
value_encrypted	TEXT	Required	Application-encrypted identifier value.
value_hash	CHAR(64)	Required	Keyed HMAC-SHA-256 used for exact duplicate detection without decrypting the value.
last_four	VARCHAR(4)	Optional	Masked display fragment used to

Column	Type	Required / Default / Key	Business meaning
			distinguish identifiers safely.
issuing_country_code	CHAR(2)	Optional	Issuing country code.
issuing_authority	VARCHAR(150)	Optional	Issuing authority.
issued_on	DATE	Optional	Calendar date for issued.
expires_on	DATE	Optional	Calendar date for expires.
verified_at	TIMESTAMPTZ	Optional	Timestamp for verified.
verified_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
is_primary	BOOLEAN	Required; Default: FALSE	Boolean flag indicating whether primary applies.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table patient_identifiers: 17 columns in the finalized SQL schema.

Key integrity and operational rules:

- Identifier values are encrypted and indexed only through keyed HMAC hashes.
- Exact duplicate identifiers are prevented.
- Only one active primary identifier is allowed per patient and type.
- Expiry and verification timestamps must be logically valid.

Indexing approach. The SQL script defines 3 explicit index(es) for this table, including 1 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

patient_addresses

Purpose. Stores one or more encrypted patient addresses and supports selecting an active primary address.

Why this table is standalone. A patient may have multiple addresses over time and one active primary address, requiring a one-to-many structure.

Key relationships. References patients, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
patient_id	UUID	Required; FK to patients; ON DELETE RESTRICT	References the related patients record.
label	VARCHAR(50)	Optional	Label.
address_line1_encrypted	TEXT	Optional	Encrypted primary address line.
address_line2_encrypted	TEXT	Optional	Encrypted secondary address line.
country_code	CHAR(2)	Optional	Country code.
region	VARCHAR(100)	Optional	Region.
district	VARCHAR(100)	Optional	District.

Column	Type	Required / Default / Key	Business meaning
ward	VARCHAR(100)	Optional	Ward.
postal_code	VARCHAR(20)	Optional	Postal code.
latitude	NUMERIC(9,6)	Optional	Latitude.
longitude	NUMERIC(10,7)	Optional	Longitude.
is_primary	BOOLEAN	Required; Default: FALSE	Boolean flag indicating whether primary applies.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table patient_addresses: 17 columns in the finalized SQL schema.

Key integrity and operational rules:

- Address details are encrypted.
- Only one active primary address is permitted per patient.
- Latitude and longitude must be provided together and within valid ranges.

Indexing approach. The SQL script defines 2 explicit index(es) for this table, including 1 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

relationship_types

Purpose. Defines reusable relationship categories such as parent, guardian, spouse, sibling, or caregiver.

Why this table is standalone. The same relationship categories are reused by many related persons and should be centrally controlled.

Key relationships. References users. Referenced by patient_related_persons.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
name	CITEXT	Required	Human-readable name.
code	VARCHAR(40)	Required	Stable business code used for identification and integration.
description	TEXT	Optional	Optional explanatory text for administrators and operators.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table relationship_types: 8 columns in the finalized SQL schema.

Key integrity and operational rules:

- Names and codes are unique.
- Codes do not automatically grant guardian or digital-access powers.

patient_related_persons

Purpose. Stores people connected to a patient for guardianship, caregiving, emergency contact, or next-of-kin purposes.

Why this table is standalone. A related person is a different human being from the patient and may carry several operational roles such as guardian and emergency contact.

Key relationships. References patients, relationship_types, users. Referenced by patient_access_grants, related_person_contacts.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
patient_id	UUID	Required; FK to patients; ON DELETE RESTRICT	References the related patients record.
relationship_type_id	UUID	Required; FK to relationship_types; ON DELETE RESTRICT	References the related relationship types record.
linked_user_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
first_name	VARCHAR(100)	Required	First name.
middle_name	VARCHAR(100)	Optional	Middle name.
last_name	VARCHAR(100)	Required	Last name.
is_guardian	BOOLEAN	Required; Default: FALSE	Boolean flag indicating whether guardian applies.
is_caregiver	BOOLEAN	Required; Default: FALSE	Boolean flag indicating whether caregiver

Column	Type	Required / Default / Key	Business meaning
			applies.
is_next_of_kin	BOOLEAN	Required; FALSE Default:	Boolean flag indicating whether next of kin applies.
is_emergency_contact	BOOLEAN	Required; FALSE Default:	Boolean flag indicating whether emergency contact applies.
priority_order	SMALLINT	Required; Default: 1	Priority order.
is_active	BOOLEAN	Required; TRUE Default:	Controls whether the record is currently available for operational use without deleting history.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table patient_related_persons: 16 columns in the finalized SQL schema.

Key integrity and operational rules:

- The linked user, when present, is unique for the patient.
- A related person may hold several operational flags.
- Guardian status does not automatically create digital access.

Indexing approach. The SQL script defines 3 explicit index(es) for this table, including 1 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

related_person_contacts

Purpose. Stores multiple phone or email contacts for each related person while preserving independent verification and primary-contact rules.

Why this table is standalone. A related person may have multiple verified phone and email values. Keeping them separate avoids repeated person rows.

Key relationships. References patient_related_persons, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
related_person_id	UUID	Required; FK to patient_related_persons; ON DELETE RESTRICT	References the related patient related persons record.
channel	VARCHAR(20)	Required	Channel.
label	VARCHAR(50)	Optional	Label.
value_encrypted	TEXT	Required	Encrypted related-person phone or email value.
value_hash	CHAR(64)	Required	Keyed HMAC used for exact-match lookup and duplicate prevention.
verified_at	TIMESTAMPTZ	Optional	Timestamp for verified.
verified_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
is_primary	BOOLEAN	Required; Default: FALSE	Boolean flag indicating whether primary applies.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for

Column	Type	Required / Default / Key	Business meaning
			operational use without deleting history.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table related_person_contacts: 13 columns in the finalized SQL schema.

Key integrity and operational rules:

- Phone/email values are encrypted and searched by HMAC hash.
- Duplicate values for the same related person and channel are prevented.
- Only one active primary contact is allowed per channel.

Indexing approach. The SQL script defines 2 explicit index(es) for this table, including 1 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

patient_access_grants

Purpose. Grants a linked related person a scoped role for one specific patient without giving broad access to all patients in an organization.

Why this table is standalone. Digital access is a revocable, time-bounded authorization decision, not an automatic consequence of being a guardian or caregiver.

Key relationships. References patient_related_persons, patients, roles, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
patient_id	UUID	Required; FK to patients; ON DELETE RESTRICT	References the related patients record.
related_person_id	UUID	Required; FK to patient_related_persons; ON DELETE RESTRICT	References the related patient related persons record.
grantee_user_id	UUID	Required; FK to users; ON DELETE RESTRICT	References the related users record.
role_id	UUID	Required; FK to roles; ON DELETE RESTRICT	Dynamic organization-scoped role whose permissions are limited to the selected patient.
granted_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
starts_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Start date and time of the period.
ends_at	TIMESTAMPTZ	Optional	End date and time of the period.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
revoked_at	TIMESTAMPTZ	Optional	Timestamp for revoked.
revoked_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.

Column	Type	Required / Default / Key	Business meaning
revocation_reason	VARCHAR(250)	Optional	Reason explaining revocation.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table patient_access_grants: 15 columns in the finalized SQL schema.

Key integrity and operational rules:

- The related person must belong to the patient and be linked to the grantee user.
- The grantee cannot be the patient's own user account.
- The role must belong to the patient organization.
- Revoked grants are immutable and cannot be silently reactivated.

Indexing approach. The SQL script defines 4 explicit index(es) for this table, including 1 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

4.4 Practitioner Registry and Professional Assignments

Maintains practitioner profiles, work locations, departments, specialties, professional credentials, and verification status.

practitioner_types

Purpose. Classifies practitioners, for example physician, nurse, clinical officer, therapist, or technician.

Why this table is standalone. Professional categories are shared across practitioners and may carry rules such as whether licensing is required.

Key relationships. References users. Referenced by practitioners.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
name	CITEXT	Required	Human-readable name.
code	VARCHAR(40)	Required	Stable business code used for identification and integration.
description	TEXT	Optional	Optional explanatory text for administrators and operators.
requires_license	BOOLEAN	Required; Default: FALSE	Boolean flag indicating whether license is required.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table practitioner_types: 9 columns in the finalized SQL schema.

Key integrity and operational rules:

- Names and codes are unique.

- `requires_license` indicates whether credential validation is expected.

practitioners

Purpose. Stores the organization-specific professional profile of a healthcare worker, independently of whether that person has a login account.

Why this table is standalone. A practitioner can exist without a system login and can work at multiple facilities, departments, and specialties.

Key relationships. References `organizations`, `practitioner_types`, `users`. Referenced by `practitioner_credentials`, `practitioner_facility_assignments`.

Column	Type	Required / Default / Key	Business meaning
<code>id</code>	UUID	Required; Default: <code>gen_random_uuid()</code> ; Primary key	System-generated UUID primary key.
<code>organization_id</code>	UUID	Required; FK to <code>organizations</code> ; ON DELETE RESTRICT	References the related <code>organizations</code> record.
<code>user_id</code>	UUID	Optional; FK to <code>users</code> ; ON DELETE SET NULL	References the related <code>users</code> record.
<code>practitioner_type_id</code>	UUID	Required; FK to <code>practitioner_types</code> ; ON DELETE RESTRICT	References the related <code>practitioner_types</code> record.
<code>practitioner_number</code>	VARCHAR(50)	Required	Organization-specific practitioner identifier.
<code>first_name</code>	VARCHAR(100)	Required	First name.
<code>middle_name</code>	VARCHAR(100)	Optional	Middle name.
<code>last_name</code>	VARCHAR(100)	Required	Last name.
<code>preferred_name</code>	VARCHAR(100)	Optional	Preferred name.
<code>email</code>	CITEXT	Optional	Normalized email address.
<code>phone_number</code>	VARCHAR(30)	Optional	Normalized E.164

Column	Type	Required / Default / Key	Business meaning
			phone number.
is_active	BOOLEAN	Required; TRUE Default:	Controls whether the record is currently available for operational use without deleting history.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table practitioners: 15 columns in the finalized SQL schema.

Key integrity and operational rules:

- Practitioner number is unique within the organization.
- A linked user may have only one practitioner profile per organization.
- Phone/email are operational contacts and are not automatically synchronized with users.

Indexing approach. The SQL script defines 4 explicit index(es) for this table, including 1 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

practitioner_facility_assignments

Purpose. Records the facilities where a practitioner is professionally assigned and the effective dates of each assignment.

Why this table is standalone. Facility work relationships are time-bounded and many-to-many between practitioners and facilities.

Key relationships. References facilities, practitioners, users. Referenced by appointments, practitioner_availability_periods, practitioner_department_assignments, practitioner_leave_requests, practitioner_shifts, practitioner_specialty_assignments.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
practitioner_id	UUID	Required; FK to practitioners; ON DELETE RESTRICT	References the related practitioners record.
facility_id	UUID	Required; FK to facilities; ON DELETE RESTRICT	References the related facilities record.
starts_on	DATE	Required	Calendar date for starts.
ends_on	DATE	Optional	Calendar date for ends.
is_primary	BOOLEAN	Required; Default: FALSE	Boolean flag indicating whether primary applies.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
assigned_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table *practitioner_facility_assignments*: 10 columns in the finalized SQL schema.

Key integrity and operational rules:

- A facility must belong to the practitioner organization.
- One assignment exists per practitioner/facility pair.
- Only one active primary facility assignment is allowed.

Indexing approach. The SQL script defines 3 explicit index(es) for this table, including 1 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

practitioner_department_assignments

Purpose. Records the departments in which a practitioner works within a facility assignment.

Why this table is standalone. A practitioner may work in several departments within one facility assignment and may change the primary department over time.

Key relationships. References departments, practitioner_facility_assignments, users. Referenced by practitioner_shifts.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
practitioner_facility_assignment_id	UUID	Required; FK to practitioner_facility_assignments; ON DELETE RESTRICT	References the related practitioner facility assignments record.
department_id	UUID	Required; FK to departments; ON DELETE RESTRICT	References the related departments record.
starts_on	DATE	Required	Calendar date for starts.
ends_on	DATE	Optional	Calendar date for ends.
is_primary	BOOLEAN	Required; Default: FALSE	Boolean flag indicating whether primary applies.
is_active	BOOLEAN	Required; Default:	Controls whether the

Column	Type	Required / Default / Key	Business meaning
		TRUE	record is currently available for operational use without deleting history.
assigned_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table practitioner_department_assignments: 10 columns in the finalized SQL schema.

Key integrity and operational rules:

- The department must belong to the facility assignment.
- Dates must remain within the facility assignment.
- Only one active primary department is allowed per facility assignment.

Indexing approach. The SQL script defines 3 explicit index(es) for this table, including 1 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

practitioner_specialty_assignments

Purpose. Records the facility specialties a practitioner is authorized to provide.

Why this table is standalone. Authorization to provide a specialty is separate from general employment at a facility and needs its own dates and primary marker.

Key relationships. References facility_specialties, practitioner_facility_assignments, users. Referenced by appointments.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
practitioner_facility_assignment_id	UUID	Required; FK to practitioner_facility_assignments; ON DELETE RESTRICT	References the related practitioner facility assignments record.
facility_specialty_id	UUID	Required; FK to facility_specialties; ON DELETE RESTRICT	References the related facility specialties record.
starts_on	DATE	Required	Calendar date for starts.
ends_on	DATE	Optional	Calendar date for ends.
is_primary	BOOLEAN	Required; Default: FALSE	Boolean flag indicating whether primary applies.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
assigned_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table practitioner_specialty_assignments: 10 columns in the finalized SQL schema.

Key integrity and operational rules:

- The facility specialty must belong to the same facility assignment.

- Department-specific specialties require a matching practitioner department assignment.
- Only one active primary specialty is allowed per facility assignment.

Indexing approach. The SQL script defines 3 explicit index(es) for this table, including 1 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

practitioner_credential_types

Purpose. Defines configurable professional credential categories such as practising licence, registration certificate, or board certification.

Why this table is standalone. Credential categories are reusable and may be global or organization-specific with different expiry requirements.

Key relationships. References organizations, users. Referenced by practitioner_credentials.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
organization_id	UUID	Optional; FK to organizations; ON DELETE RESTRICT	References the related organizations record.
name	VARCHAR(100)	Required	Human-readable name.
code	VARCHAR(40)	Required	Stable business code used for identification and integration.
description	TEXT	Optional	Optional explanatory text for administrators and operators.
country_code	CHAR(2)	Optional	Country code.
requires_expiry_date	BOOLEAN	Required; Default: FALSE	Boolean flag indicating whether expiry date is required.
requires_verification	BOOLEAN	Required; Default: TRUE	Boolean flag indicating whether verification is

Column	Type	Required / Default / Key	Business meaning
			required.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table *practitioner_credential_types*: 12 columns in the finalized SQL schema.

Key integrity and operational rules:

- Types may be global or organization-specific.
- Expiry and verification requirements are configurable.
- Codes and names are unique within scope.

Indexing approach. The SQL script defines 5 explicit index(es) for this table, including 4 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

practitioner_credentials

Purpose. Stores encrypted credential numbers, issuing authorities, expiry dates, and verification decisions for practitioners.

Why this table is standalone. Credentials are sensitive, independently verified, expirable records that may occur several times per practitioner.

Key relationships. References practitioner_credential_types, practitioners, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
practitioner_id	UUID	Required; FK to practitioners; ON DELETE RESTRICT	References the related practitioners record.
credential_type_id	UUID	Required; FK to practitioner_credential_types; ON DELETE RESTRICT	References the related practitioner credential types record.
credential_number_encrypted	TEXT	Required	Encrypted professional credential or licence number.
credential_number_hash	CHAR(64)	Required	Keyed HMAC used to identify duplicate credential numbers securely.
last_four	VARCHAR(4)	Optional	Last four.
issuing_authority	VARCHAR(150)	Optional	Issuing authority.
issuing_country_code	CHAR(2)	Optional	Issuing country code.
issued_on	DATE	Optional	Calendar date for issued.
expires_on	DATE	Optional	Calendar date for expires.
verification_status	VARCHAR(20)	Required; Default: 'unverified'	Current status of verification.
verified_at	TIMESTAMPTZ	Optional	Timestamp for verified.
verified_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
is_active	BOOLEAN	Required; Default:	Controls whether the

Column	Type	Required / Default / Key	Business meaning
		TRUE	record is currently available for operational use without deleting history.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table practitioner_credentials: 17 columns in the finalized SQL schema.

Key integrity and operational rules:

- Credential values are encrypted and HMAC-indexed.
- Credential type scope must match the practitioner organization.
- Verified or rejected records require reviewer and decision time.
- Expiry rules follow the credential type.

Indexing approach. The SQL script defines 2 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

4.5 Scheduling and Appointment Management

Controls recurring availability, leave, shifts, rooms, bookable slots, appointment conflicts, rescheduling, and lifecycle history.

practitioner_availability_periods

Purpose. Stores the practitioner's recurring weekly availability for appointment scheduling at a facility.

Why this table is standalone. Recurring weekly availability is one-to-many and differs from actual dated shifts and approved leave.

Key relationships. References practitioner_facility_assignments, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
practitioner_facility_assignment_id	UUID	Required; FK to practitioner_facility_assignments; ON DELETE RESTRICT	References the related practitioner facility assignments record.
day_of_week	SMALLINT	Required	Day of week.
starts_at	TIME	Required	Start date and time of the period.
ends_at	TIME	Required	End date and time of the period.
valid_from	DATE	Required	Date from which the record becomes effective.
valid_until	DATE	Optional	Optional final date on which the record remains effective.
is_available_for_appointments	BOOLEAN	Required; Default: TRUE	Boolean flag indicating whether available for appointments applies.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.

Column	Type	Required / Default / Key	Business meaning
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table practitioner_availability_periods: 12 columns in the finalized SQL schema.

Key integrity and operational rules:

- Periods use facility-local weekly time and a valid date range.
- Active periods cannot overlap for the same practitioner assignment.
- Availability is checked together with assignment state, leave, shifts, and exceptions.

Indexing approach. The SQL script defines 2 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

practitioner_leave_requests

Purpose. Stores practitioner absence requests and approval decisions that block scheduling when approved.

Why this table is standalone. Leave has a request/decision/cancellation workflow and must remain historically traceable after it affects schedules.

Key relationships. References practitioner_facility_assignments, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
practitioner_facility_assignment_id	UUID	Required; FK to practitioner_facility_assignments; ON DELETE RESTRICT	References the related practitioner facility assignments record.

Column	Type	Required / Default / Key	Business meaning
starts_at	TIMESTAMPTZ	Required	Start date and time of the period.
ends_at	TIMESTAMPTZ	Required	End date and time of the period.
reason	VARCHAR(250)	Optional	Reason or explanation for the record or transition.
status	VARCHAR(20)	Required; Default: 'pending'	Current operational workflow state.
requested_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
decided_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
decided_at	TIMESTAMPTZ	Optional	Timestamp for decided.
decision_note	VARCHAR(250)	Optional	Decision note.
cancelled_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
cancelled_at	TIMESTAMPTZ	Optional	Timestamp for cancelled.
cancellation_reason	VARCHAR(250)	Optional	Reason explaining cancellation.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table practitioner_leave_requests: 15 columns in the finalized SQL schema.

Key integrity and operational rules:

- Only approved leave blocks scheduling.
- Pending and approved leave periods cannot overlap.
- Approval/rejection and cancellation fields must match the workflow state.

Indexing approach. The SQL script defines 1 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

practitioner_shifts

Purpose. Stores actual dated work periods, assigned department, service point, consultation room, and actual start/end performance.

Why this table is standalone. A shift is a dated operational commitment with actual timing and room allocation, unlike recurring availability.

Key relationships. References consultation_rooms, practitioner_department_assignments, practitioner_facility_assignments, service_points, users. Referenced by appointment_slots, appointments, queue_entries.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
practitioner_facility_assignment_id	UUID	Required; FK to practitioner_facility_assignments; ON DELETE RESTRICT	References the related practitioner facility assignments record.
practitioner_department_assignment_id	UUID	Optional; FK to practitioner_department_assignments; ON DELETE RESTRICT	References the related practitioner department assignments record.
service_point_id	UUID	Optional; FK to service_points; ON DELETE RESTRICT	References the related service points record.
consultation_room_id	UUID	Optional; FK to	Optional room reserved

Column	Type	Required / Default / Key	Business meaning
		consultation_rooms; ON DELETE RESTRICT	for the shift; protected against overlapping allocations.
starts_at	TIMESTAMPTZ	Required	Start date and time of the period.
ends_at	TIMESTAMPTZ	Required	End date and time of the period.
actual_started_at	TIMESTAMPTZ	Optional	Actual time the practitioner began the shift.
actual_ended_at	TIMESTAMPTZ	Optional	Actual time the practitioner completed the shift.
accepts_appointments	BOOLEAN	Required; Default: TRUE	Boolean flag controlling whether the record accepts appointments.
status	VARCHAR(20)	Required; Default: 'scheduled'	Current operational workflow state.
notes	VARCHAR(250)	Optional	Optional short operational note.
cancelled_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
cancelled_at	TIMESTAMPTZ	Optional	Timestamp for cancelled.
cancellation_reason	VARCHAR(250)	Optional	Reason explaining cancellation.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default:	Timestamp when the

Column	Type	Required / Default / Key	Business meaning
		CURRENT_TIMESTAMP	record was created.
updated_at	TIMESTAMPZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table practitioner_shifts: 18 columns in the finalized SQL schema.

Key integrity and operational rules:

- Shift must fall within active facility and department assignments.
- Practitioner, room, and service-point conflicts are rejected.
- Cancelled, in-progress, and completed statuses require consistent timestamps.
- Appointment booking may use only shifts that accept appointments.

Indexing approach. The SQL script defines 5 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

appointments

Purpose. Stores patient bookings, schedule, specialty, optional practitioner/shift/slot assignment, booking source, and current lifecycle state.

Why this table is standalone. Appointments are durable commercial transactions with their own number, lifecycle, rescheduling chain, privacy requirements, and conflict rules.

Key relationships. References appointments, facilities, facility_specialties, patients, practitioner_facility_assignments, practitioner_shifts, practitioner_specialty_assignments, users. Referenced by appointment_status_history, appointments, checkin_tokens, patient_checkins, patient_notifications.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.

Column	Type	Required / Default / Key	Business meaning
facility_id	UUID	Required; FK to facilities; ON DELETE RESTRICT	References the related facilities record.
patient_id	UUID	Required; FK to patients; ON DELETE RESTRICT	References the related patients record.
facility_specialty_id	UUID	Required; FK to facility_specialties; ON DELETE RESTRICT	References the related facility specialties record.
practitioner_facility_assignment_id	UUID	Optional; FK to practitioner_facility_assignments; ON DELETE RESTRICT	References the related practitioner facility assignments record.
practitioner_specialty_assignment_id	UUID	Optional; FK to practitioner_specialty_assignments; ON DELETE RESTRICT	References the related practitioner specialty assignments record.
practitioner_shift_id	UUID	Optional; FK to practitioner_shifts; ON DELETE RESTRICT	References the related practitioner shifts record.
appointment_slot_id	UUID	Optional	Identifier of the related appointment slot record.
appointment_number	VARCHAR(50)	Required	Facility-specific human-readable appointment reference.
scheduled_start	TIMESTAMPTZ	Required	Planned appointment start timestamp.
scheduled_end	TIMESTAMPTZ	Required	Planned appointment end timestamp.
status	VARCHAR(20)	Required; Default: 'pending'	Current operational workflow state.
booking_channel	VARCHAR(20)	Required	Origin of the booking: mobile, web, reception,

Column	Type	Required / Default / Key	Business meaning
			or API.
reason_for_visit_encrypted	TEXT	Optional	Encrypted optional reason for visit because it may contain health information.
rescheduled_from_id	UUID	Optional; FK to appointments; ON DELETE RESTRICT	Previous appointment replaced by this rescheduled booking.
cancelled_at	TIMESTAMPTZ	Optional	Timestamp for cancelled.
cancelled_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
cancellation_reason	VARCHAR(250)	Optional	Reason explaining cancellation.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table appointments: 21 columns in the finalized SQL schema.

Key integrity and operational rules:

- Appointment number is unique within the facility.
- Patient, specialty, practitioner, shift, and slot must all match the same organization/facility context.
- Active practitioner and patient appointments cannot overlap.
- Rescheduling creates a new appointment and preserves the original row.

Indexing approach. The SQL script defines 8 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

appointment_status_history

Purpose. Preserves every appointment lifecycle transition as an append-only operational history.

Why this table is standalone. A single current status cannot explain when or why transitions occurred. Append-only history supports compliance, reporting, and troubleshooting.

Key relationships. References appointments, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
appointment_id	UUID	Required; FK to appointments; ON DELETE RESTRICT	References the related appointments record.
from_status	VARCHAR(20)	Optional	Appointment status before the transition; null only for the initial event.
to_status	VARCHAR(20)	Required	Appointment status after the transition.
change_source	VARCHAR(20)	Required	System surface that initiated the transition.
changed_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
reason	VARCHAR(250)	Optional	Reason or explanation for the record or transition.
changed_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp for changed.

Table *appointment_status_history*: 8 columns in the finalized SQL schema.

Key integrity and operational rules:

- Rows are append-only.
- Only the initial record may have a null previous status.
- The latest destination status must match appointments.status.
- Status update and history insertion occur in one transaction.

Indexing approach. The SQL script defines 3 explicit index(es) for this table, including 1 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

appointment_slots

Purpose. Represents concrete bookable time intervals within practitioner shifts and controls slot capacity safely under concurrent booking.

Why this table is standalone. Slots are repeated bookable capacity units. Their concurrent reservation counter and status cannot be stored on the shift itself.

Key relationships. References facility_specialties, practitioner_shifts.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
practitioner_shift_id	UUID	Required; FK to practitioner_shifts; ON DELETE RESTRICT	References the related practitioner shifts record.
facility_specialty_id	UUID	Required; FK to facility_specialties; ON DELETE RESTRICT	References the related facility specialties record.
starts_at	TIMESTAMPTZ	Required	Start date and time of the period.
ends_at	TIMESTAMPTZ	Required	End date and time of the period.
capacity	SMALLINT	Required; Default: 1	Maximum active appointments allowed

Column	Type	Required / Default / Key	Business meaning
			in the slot.
booked_count	SMALLINT	Required; Default: 0	Transactionally maintained count of active reservations.
status	VARCHAR(20)	Required; Default: 'available'	Current operational workflow state.
is_online_bookable	BOOLEAN	Required; Default: TRUE	Boolean flag indicating whether online bookable applies.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table appointment_slots: 11 columns in the finalized SQL schema.

Key integrity and operational rules:

- Slot lies completely inside its practitioner shift.
- Capacity is positive and booked_count remains between zero and capacity.
- Booking locks the slot row to prevent concurrent overbooking.
- Blocked, full, or cancelled slots reject new appointments.

Indexing approach. The SQL script defines 3 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

4.6 Patient Arrival and Secure Check-in

Records physical or digital arrival and secures QR check-in through expiring single-use tokens.

patient_checkins

Purpose. Records patient arrival for either an appointment or a walk-in request and supports reception, mobile, QR, or self-service check-in.

Why this table is standalone. Arrival is a separate real-world event from booking. It also supports walk-ins who have no appointment.

Key relationships. References appointments, facilities, facility_specialties, patients, users. Referenced by checkin_tokens, queue_entries.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
facility_id	UUID	Required; FK to facilities; ON DELETE RESTRICT	References the related facilities record.
patient_id	UUID	Required; FK to patients; ON DELETE RESTRICT	References the related patients record.
appointment_id	UUID	Optional; FK to appointments; ON DELETE RESTRICT	References the related appointments record.
facility_specialty_id	UUID	Optional; FK to facility_specialties; ON DELETE RESTRICT	References the related facility specialties record.
checkin_method	VARCHAR(20)	Required	Arrival method: reception, mobile, QR code, or self-service.
checked_in_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Actual arrival confirmation timestamp.
checked_in_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.

Column	Type	Required / Default / Key	Business meaning
notes	VARCHAR(250)	Optional	Optional short operational note.
voided_at	TIMESTAMPTZ	Optional	Timestamp for voided.
voided_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
void_reason	VARCHAR(250)	Optional	Reason explaining void.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table patient_checkins: 14 columns in the finalized SQL schema.

Key integrity and operational rules:

- Appointment check-in must match the patient and facility.
- Walk-in check-in requires a facility specialty that accepts walk-ins.
- Only one active check-in is allowed per appointment.
- Voided check-ins remain historically visible.

Indexing approach. The SQL script defines 4 explicit index(es) for this table, including 1 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

checkin_tokens

Purpose. Stores secure, short-lived, single-use QR check-in token hashes for appointments.

Why this table is standalone. QR tokens have independent security, expiry, revocation, and single-use rules and should never be stored on appointments.

Key relationships. References appointments, patient_checkins, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
appointment_id	UUID	Required; FK to appointments; ON DELETE RESTRICT	References the related appointments record.
token_hash	CHAR(64)	Required	HMAC-SHA-256 of the single-use raw QR token; the raw token is never stored.
expires_at	TIMESTAMPTZ	Required	Final time at which the token may be consumed.
used_at	TIMESTAMPTZ	Optional	Timestamp for used.
patient_checkin_id	UUID	Optional; FK to patient_checkins; ON DELETE RESTRICT	References the related patient checkins record.
revoked_at	TIMESTAMPTZ	Optional	Timestamp for revoked.
revoked_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
revocation_reason	VARCHAR(250)	Optional	Reason explaining revocation.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.

Table checkin_tokens: 11 columns in the finalized SQL schema.

Key integrity and operational rules:

- Only HMAC token hashes are stored.

- Only one unused, non-revoked token is allowed per appointment.
- Tokens are single-use, expirable, and consumed under row lock.
- The token and resulting check-in must reference the same appointment.

Indexing approach. The SQL script defines 4 explicit index(es) for this table, including 2 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

4.7 Queue Operations

Manages daily queues, priority-aware patient entries, calls, service timestamps, transfer chains, and immutable event history.

queues

Purpose. Represents a daily operational queue for a service point and optionally a facility specialty.

Why this table is standalone. The queue is the daily container and sequence generator; patient entries are separate children.

Key relationships. References facility_specialties, service_points, users. Referenced by queue_entries.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
service_point_id	UUID	Required; FK to service_points; ON DELETE RESTRICT	References the related service points record.
facility_specialty_id	UUID	Optional; FK to facility_specialties; ON DELETE RESTRICT	References the related facility specialties record.
queue_date	DATE	Required	Facility-local business date of the queue.
next_sequence_number	INTEGER	Required; Default: 1	Next queue sequence issued under a row-level lock.
status	VARCHAR(20)	Required; Default:	Current operational

Column	Type	Required / Default / Key	Business meaning
		'draft'	workflow state.
opened_at	TIMESTAMPTZ	Optional	Timestamp for opened.
opened_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
paused_at	TIMESTAMPTZ	Optional	Timestamp for paused.
closed_at	TIMESTAMPTZ	Optional	Timestamp for closed.
closed_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table queues: 14 columns in the finalized SQL schema.

Key integrity and operational rules:

- One daily queue exists per service point and optional specialty context.
- Sequence generation locks the queue row.
- Only open queues accept new entries.
- Displayed prefixes come from service_points.code and are not duplicated.

Indexing approach. The SQL script defines 4 explicit index(es) for this table, including 2 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

queue_entries

Purpose. Represents each checked-in patient waiting or being served in a queue, including priority, sequence, practitioner assignment, and service timestamps.

Why this table is standalone. Each patient has independent priority, timestamps, status, and optional practitioner assignment within the queue.

Key relationships. References patient_checkins, practitioner_shifts, queues, users. Referenced by patient_notifications, queue_entry_events, queue_transfers, queue_wait_time_predictions.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
queue_id	UUID	Required; FK to queues; ON DELETE RESTRICT	References the related queues record.
patient_checkin_id	UUID	Required; FK to patient_checkins; ON DELETE RESTRICT	References the related patient checkins record.
practitioner_shift_id	UUID	Optional; FK to practitioner_shifts; ON DELETE RESTRICT	References the related practitioner shifts record.
sequence_number	INTEGER	Required	Daily number assigned within the queue.
priority_level	SMALLINT	Required; Default: 0	Service priority from 0 normal to 3 emergency.
priority_reason	VARCHAR(250)	Optional	Reason explaining priority.
status	VARCHAR(20)	Required; Default: 'waiting'	Current operational workflow state.
joined_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Time the patient entered the queue.
called_at	TIMESTAMPTZ	Optional	First time the patient

Column	Type	Required / Default / Key	Business meaning
			was called.
service_started_at	TIMESTAMPTZ	Optional	Actual time service began.
service_completed_at	TIMESTAMPTZ	Optional	Actual time service ended.
cancelled_at	TIMESTAMPTZ	Optional	Timestamp for cancelled.
cancelled_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
cancellation_reason	VARCHAR(250)	Optional	Reason explaining cancellation.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table queue_entries: 18 columns in the finalized SQL schema.

Key integrity and operational rules:

- Sequence number is unique within the queue.
- The same check-in cannot enter the same queue twice.
- Priority is limited to 0-3 and affects ordering, not the issued number.
- Status and service timestamps must remain consistent.

Indexing approach. The SQL script defines 4 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

queue_transfers

Purpose. Preserves movement from one queue entry to another so the original queue history and destination queue number remain traceable.

Why this table is standalone. Transfer is a relationship between two entries and must preserve source and destination history atomically.

Key relationships. References queue_entries, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
source_queue_entry_id	UUID	Required; FK to queue_entries; ON DELETE RESTRICT	Original queue entry being closed as transferred.
destination_queue_entry_id	UUID	Required; FK to queue_entries; ON DELETE RESTRICT	New queue entry created in the destination queue.
transferred_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
transfer_reason	VARCHAR(250)	Required	Reason explaining transfer.
transferred_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp for transferred.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.

Table queue_transfers: 7 columns in the finalized SQL schema.

Key integrity and operational rules:

- Source and destination entries must represent the same check-in.
- Source and destination queues must differ but remain in the same facility.
- The transfer, destination entry, source status, and sequence increment occur atomically.

Indexing approach. The SQL script defines 1 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

queue_entry_events

Purpose. Stores append-only queue lifecycle events, including repeated calls that cannot be represented by one current-state row.

Why this table is standalone. Repeated calls and lifecycle transitions require multiple immutable events per queue entry.

Key relationships. References queue_entries, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
queue_entry_id	UUID	Required; FK to queue_entries; ON DELETE RESTRICT	References the related queue entries record.
event_type	VARCHAR(30)	Required	Operational event such as called, recalled, skipped, service_started, or priority_changed.
from_status	VARCHAR(20)	Optional	Current status of from.
to_status	VARCHAR(20)	Optional	Current status of to.
performed_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
reason	VARCHAR(250)	Optional	Reason or explanation for the record or

Column	Type	Required / Default / Key	Business meaning
			transition.
occurred_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp for occurred.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.

Table queue_entry_events: 9 columns in the finalized SQL schema.

Key integrity and operational rules:

- Rows are append-only.
- Repeated calls are recorded as separate events.
- The latest applicable event status must match queue_entries.status.
- Reasons are required for cancellation, transfer, and priority changes.

Indexing approach. The SQL script defines 3 explicit index(es) for this table, including 1 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

4.8 Intelligence and Prediction

Stores repeated waiting-time estimates and model versions so prediction quality can be evaluated against actual waits.

queue_wait_time_predictions

Purpose. Stores successive rule-based or machine-learning waiting-time predictions for each queue entry.

Why this table is standalone. Predictions change over time and need versioned history for model evaluation; storing only the latest value would destroy evidence.

Key relationships. References queue_entries.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default:	System-generated

Column	Type	Required / Default / Key	Business meaning
		gen_random_uuid(); Primary key	UUID primary key.
queue_entry_id	UUID	Required; FK to queue_entries; ON DELETE RESTRICT	References the related queue entries record.
predicted_wait_minutes	INTEGER	Required	Estimated remaining waiting time in minutes.
prediction_method	VARCHAR(30)	Required	Indicates rule-based or machine-learning prediction.
model_version	VARCHAR(100)	Optional	Version identifier for the rule set or trained model.
confidence_score	NUMERIC(5,4)	Optional	Optional model confidence value between 0 and 1.
generated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp for generated.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.

Table queue_wait_time_predictions: 8 columns in the finalized SQL schema.

Key integrity and operational rules:

- Predictions are append-only and may be regenerated while waiting.
- Machine-learning predictions require a model version.
- Confidence score is limited to 0-1.
- Actual wait is derived from queue timestamps rather than stored redundantly.

Indexing approach. The SQL script defines 2 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

4.9 Patient Communication

Schedules and tracks encrypted multi-channel notifications and manages push-notification devices.

patient_notifications

Purpose. Stores scheduled and delivered patient communications across SMS, email, push, and in-app channels.

Why this table is standalone. Each channel delivery has its own schedule, destination snapshot, retry state, provider reference, and final outcome.

Key relationships. References appointments, patients, queue_entries, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
patient_id	UUID	Required; FK to patients; ON DELETE RESTRICT	References the related patients record.
appointment_id	UUID	Optional; FK to appointments; ON DELETE RESTRICT	References the related appointments record.
queue_entry_id	UUID	Optional; FK to queue_entries; ON DELETE RESTRICT	References the related queue entries record.
notification_type	VARCHAR(40)	Required	Notification type.
channel	VARCHAR(20)	Required	Channel.
recipient_user_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
destination_encrypted	TEXT	Optional	Encrypted phone number, email address, or push destination snapshot.
subject_encrypted	TEXT	Optional	Application-encrypted

Column	Type	Required / Default / Key	Business meaning
			value for subject.
body_encrypted	TEXT	Required	Encrypted fully rendered notification body.
scheduled_for	TIMESTAMPTZ	Required	Scheduled for.
status	VARCHAR(20)	Required; Default: 'pending'	Current operational workflow state.
attempt_count	SMALLINT	Required; Default: 0	Stored count of attempt.
last_attempt_at	TIMESTAMPTZ	Optional	Timestamp for last attempt.
sent_at	TIMESTAMPTZ	Optional	Timestamp for sent.
delivered_at	TIMESTAMPTZ	Optional	Timestamp for delivered.
read_at	TIMESTAMPTZ	Optional	Timestamp for read.
failed_at	TIMESTAMPTZ	Optional	Timestamp for failed.
failure_reason	VARCHAR(250)	Optional	Reason explaining failure.
provider_message_id	VARCHAR(150)	Optional	Identifier of the related provider message record.
idempotency_key	VARCHAR(100)	Optional	Unique application key that prevents duplicate scheduling.
created_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	User who created the record, retained as nullable history if the user is later removed.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default:	Timestamp of the most

Column	Type	Required / Default / Key	Business meaning
		CURRENT_TIMESTAMP	recent update; maintained automatically where applicable.

Table patient_notifications: 24 columns in the finalized SQL schema.

Key integrity and operational rules:

- One row represents one channel delivery.
- Destination and rendered content are encrypted.
- Idempotency keys prevent duplicate scheduled messages.
- Delivery, failure, cancellation, and read timestamps must match status.

Indexing approach. The SQL script defines 6 explicit index(es) for this table, including 1 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

user_push_devices

Purpose. Stores encrypted push-notification tokens for each user device or browser endpoint.

Why this table is standalone. A user may have many devices, each with an independent token, revocation state, and last-seen time.

Key relationships. References users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
user_id	UUID	Required; FK to users; ON DELETE RESTRICT	References the related users record.
platform	VARCHAR(15)	Required	Platform.
token_encrypted	TEXT	Required	Encrypted provider push token.

Column	Type	Required / Default / Key	Business meaning
token_hash	CHAR(64)	Required	HMAC-SHA-256 used to locate and deduplicate the token safely.
device_name	VARCHAR(100)	Optional	Device name.
app_version	VARCHAR(30)	Optional	App version.
last_seen_at	TIMESTAMPTZ	Optional	Timestamp for last seen.
is_active	BOOLEAN	Required; Default: TRUE	Controls whether the record is currently available for operational use without deleting history.
revoked_at	TIMESTAMPTZ	Optional	Timestamp for revoked.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.
updated_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table *user_push_devices*: 12 columns in the finalized SQL schema.

Key integrity and operational rules:

- Token values are encrypted and globally deduplicated by HMAC hash.
- Revoked devices are inactive.
- Raw tokens are never logged.

Indexing approach. The SQL script defines 1 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

4.10 Audit and Reporting

Provides immutable platform audit evidence and tracks secure asynchronous report generation.

audit_logs

Purpose. Stores append-only security and administrative audit records across the whole platform.

Why this table is standalone. Audit evidence spans all domains and must remain immutable without altering business tables.

Key relationships. References facilities, organizations, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
organization_id	UUID	Optional; FK to organizations; ON DELETE RESTRICT	References the related organizations record.
facility_id	UUID	Optional; FK to facilities; ON DELETE RESTRICT	References the related facilities record.
actor_user_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
action	VARCHAR(50)	Required	Audited action code, such as appointment.cancelled.
entity_type	VARCHAR(100)	Required	Business entity type affected by the action.
entity_id	UUID	Optional	Identifier of the affected entity when available.
source	VARCHAR(20)	Required	Source.
request_id	UUID	Optional	Correlation identifier used to trace a request across services and

Column	Type	Required / Default / Key	Business meaning
			logs.
ip_address	INET	Optional	Ip address.
user_agent	VARCHAR(500)	Optional	User agent.
changes	JSONB	Optional	Redacted before/after change summary.
metadata	JSONB	Optional	Redacted structured context that supports investigation without storing sensitive payloads.
occurred_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp for occurred.
created_at	TIMESTAMPTZ	Required; Default: CURRENT_TIMESTAMP	Timestamp when the record was created.

Table audit_logs: 15 columns in the finalized SQL schema.

Key integrity and operational rules:

- Rows are append-only.
- Sensitive values are redacted from changes and metadata.
- System actions may have no actor user.
- Audit logs complement rather than replace domain history tables.

Indexing approach. The SQL script defines 5 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

report_exports

Purpose. Tracks asynchronous generation, storage, expiry, and failure of CSV, Excel, and PDF report files.

Why this table is standalone. Report generation is asynchronous and has its own status, private storage, expiry, and failure lifecycle.

Key relationships. References facilities, organizations, users.

Column	Type	Required / Default / Key	Business meaning
id	UUID	Required; Default: gen_random_uuid(); Primary key	System-generated UUID primary key.
organization_id	UUID	Required; FK to organizations; ON DELETE RESTRICT	References the related organizations record.
facility_id	UUID	Optional; FK to facilities; ON DELETE RESTRICT	References the related facilities record.
report_type	VARCHAR(50)	Required	Report type.
export_format	VARCHAR(10)	Required	Export format.
parameters	JSONB	Optional	Redacted structured filters used to generate the report.
status	VARCHAR(20)	Required; Default: 'pending'	Current operational workflow state.
requested_by_id	UUID	Optional; FK to users; ON DELETE SET NULL	References the related users record.
storage_key	VARCHAR(500)	Optional	Private object-storage reference for the completed export.
row_count	INTEGER	Optional	Stored count of row.
generated_at	TIMESTAMPTZ	Optional	Timestamp for generated.
expires_at	TIMESTAMPTZ	Optional	Timestamp for expires.
failed_at	TIMESTAMPTZ	Optional	Timestamp for failed.
failure_reason	VARCHAR(250)	Optional	Reason explaining failure.
created_at	TIMESTAMPTZ	Required; Default:	Timestamp when the

Column	Type	Required / Default / Key	Business meaning
		CURRENT_TIMESTAMP	record was created.
updated_at	TIMESTAMPZ	Required; Default: CURRENT_TIMESTAMP	Timestamp of the most recent update; maintained automatically where applicable.

Table report_exports: 16 columns in the finalized SQL schema.

Key integrity and operational rules:

- Facility must belong to the selected organization.
- Completed exports require a private storage key and generation time.
- Failed exports require failure time and reason.
- Storage keys are private references, not permanent public URLs.

Indexing approach. The SQL script defines 3 explicit index(es) for this table, including 0 unique or partial-unique index(es). Indexes target foreign keys, active-status filters, dates, and common operational lookup paths.

5. Critical Transaction Workflows

Appointment booking

1. Validate patient, facility, specialty, practitioner assignment, availability, leave, shift, room, slot, and active conflicts.
2. Lock the appointment slot when a predefined slot is used.
3. Create the appointment and update booked_count in the same transaction.
4. Insert the initial appointment_status_history row before commit.

Appointment rescheduling

5. Keep the original appointment unchanged except for status = rescheduled.
6. Create a new appointment linked through rescheduled_from_id.
7. Release the old slot capacity and reserve the new slot atomically.
8. Schedule a rescheduling notification through an idempotent key.

QR check-in

9. Lock the checkin_tokens row and compare the HMAC hash.
10. Reject expired, revoked, used, or already-checked-in appointments.
11. Create patient_checkins, update appointment status/history, mark the token used, and create the queue entry in one transaction.

Queue entry creation

12. Lock the queues row using SELECT FOR UPDATE.
13. Read next_sequence_number and create queue_entries.
14. Increment next_sequence_number and commit together so duplicate numbers cannot occur.

Queue service progression

15. Lock queue_entries before each transition.
16. Update the current status/timestamps and append queue_entry_events.
17. Update linked appointment status and appointment_status_history in the same transaction.

Queue transfer

18. Lock the source queue entry and destination queue.
19. Create the destination entry with a new destination sequence.
20. Mark the source transferred and create queue_transfers atomically.

Notification delivery

21. Claim pending notifications using a worker-safe locking strategy.
22. Increment attempt_count and record provider references without exposing message content.
23. Use idempotency_key so retries do not create duplicate business notifications.

6. Security, Privacy, and Audit

6.1 Sensitive Data Protection

- Encrypt patient identifiers, addresses, practitioner credential numbers, appointment reasons, notification destinations/content, and push tokens before database insertion.
- Use a separate encryption key management service or environment-managed key hierarchy; do not store encryption keys in the database.
- Use keyed HMAC-SHA-256 for exact-match lookup. Never use plain SHA-256 for low-entropy identifiers such as phone numbers or national IDs.
- Mask sensitive display values using last_four or equivalent safe fragments.
- Do not place decrypted values in application logs, audit metadata, Celery task arguments, error traces, or analytics events.

6.2 Authorization Boundaries

- Platform, organization, and facility roles are dynamic records, not code constants.
- Organization membership is validated before organization roles are assigned.
- Facility membership is validated before facility roles are assigned.
- Patient-specific delegated access uses patient_access_grants and does not automatically follow family relationship flags.
- Django is_staff controls access to Django administration only; it is not a substitute for business roles.

6.3 Audit and Immutability

- `appointment_status_history`, `queue_entry_events`, `queue_wait_time_predictions`, and `audit_logs` are append-only.
- Operational lifecycle tables answer what happened in the domain; `audit_logs` answer who performed or attempted an action.
- Audit changes and metadata must be redacted before insertion.
- Database access should use separate application, migration, reporting, and administrative roles following least privilege.

7. Performance and Reporting Strategy

The SQL script contains explicit indexes for foreign keys, status/date combinations, partial uniqueness, current active records, and high-frequency operational paths. Exclusion constraints protect time-range conflicts. The initial deployment should use live queries and carefully selected views; materialized summaries should be introduced only after measured performance requires them.

- Do not store queue position; calculate it from active entries ordered by priority, joined time, and sequence.
- Do not store actual waiting time; calculate `service_started_at` minus `joined_at`.
- Do not store dashboard totals on operational rows; aggregate with indexed queries or reporting views.
- Use read replicas or a reporting database only when production load justifies the operational complexity.
- Review query plans with `EXPLAIN ANALYZE` before adding additional indexes.
- Partition `audit_logs` and high-volume event/prediction tables by time only when data volume proves the need.

8. Django and Deployment Guidance

- Set `Meta.db_table` on every Django model so table names match this document and SQL script exactly.
- Represent complex exclusion constraints, partial unique indexes, and database triggers through explicit migrations rather than relying only on model validation.

- Do not execute the full SQL script over a database whose tables were already created by unrelated migrations. Use one authoritative migration strategy.
- Run all booking, check-in, queue-number, transfer, and status-history workflows through service-layer transactions.
- Use SELECT FOR UPDATE for mutable counters and single-use tokens.
- Keep PostgreSQL and application time in UTC; convert to the facility timezone at the boundary.
- Back up both the database and encryption-key material according to separate secure procedures.

8.1 Recommended Application Modules

Django app	Responsibility
accounts	users, roles, permissions, memberships, and role assignments
facilities	organizations, facilities, departments, specialties, service points, rooms, and settings
patients	patient profiles, identifiers, addresses, related persons, and access grants
practitioners	practitioner profiles, assignments, specialties, and credentials
scheduling	availability, leave, shifts, slots, appointments, and status history
checkins	patient arrival and QR token handling
queueing	queues, entries, transfers, and queue events
intelligence	waiting-time prediction creation and evaluation
notifications	notifications and push-device management
audit	append-only audit capture and query services
reporting	report queries and export lifecycle

9. Deliberately Excluded Tables

The following tables were intentionally not created because the current commercial scope does not yet justify their maintenance cost:

- patient_contacts and practitioner_contacts - one phone and email are sufficient on the owner rows.
- notification_templates - templates can initially be versioned in application configuration.
- notification_delivery_attempts - the current notification table stores aggregate attempt state; add detailed attempts only when provider-level audit is required.
- report_schedules - recurring report delivery is not part of the current scope.
- ML model registry - model_version on predictions is sufficient for the first deployment.
- dashboard_statistics, daily_queue_summaries, and workload_summaries - these are derived reporting outputs, not transaction sources.
- generic_system_settings JSON table - explicit typed settings are safer and easier to validate.

10. Conclusion

The finalized schema provides a clean commercial foundation for appointment scheduling and patient flow. It protects tenant boundaries, preserves operational history, supports secure patient and practitioner data, prevents common scheduling conflicts, and provides reliable source data for waiting-time prediction and reporting. The design is intentionally extensible, but new tables should be added only when a confirmed one-to-many relationship, independent lifecycle, security requirement, or historical requirement exists.

Appendix A. Source SQL Script

The authoritative PostgreSQL implementation file is `intelligent_patient_flow_database.sql`. This documentation explains the business purpose and usage of that schema; the SQL file remains the executable technical definition for extensions, tables, constraints, indexes, functions, and triggers.